

CSE 142 Homework 5, Spring 2025

60 pts, due: June 5th, 2025

Your name: Liam Youm

Your Username: tyoun@ucsc.edu

Instruction

- Submit your assignments onto **Canvas** by the due date. Upload a `zip` file containing:

(1) The saved/latest `.ipynb` file, please **rename this file with your name included**.

(2) Also save your file into a pdf version, if error appears, save an html version instead (easier to grade for written questions).

For assignment related questions, please reach the course staff through discord or email.

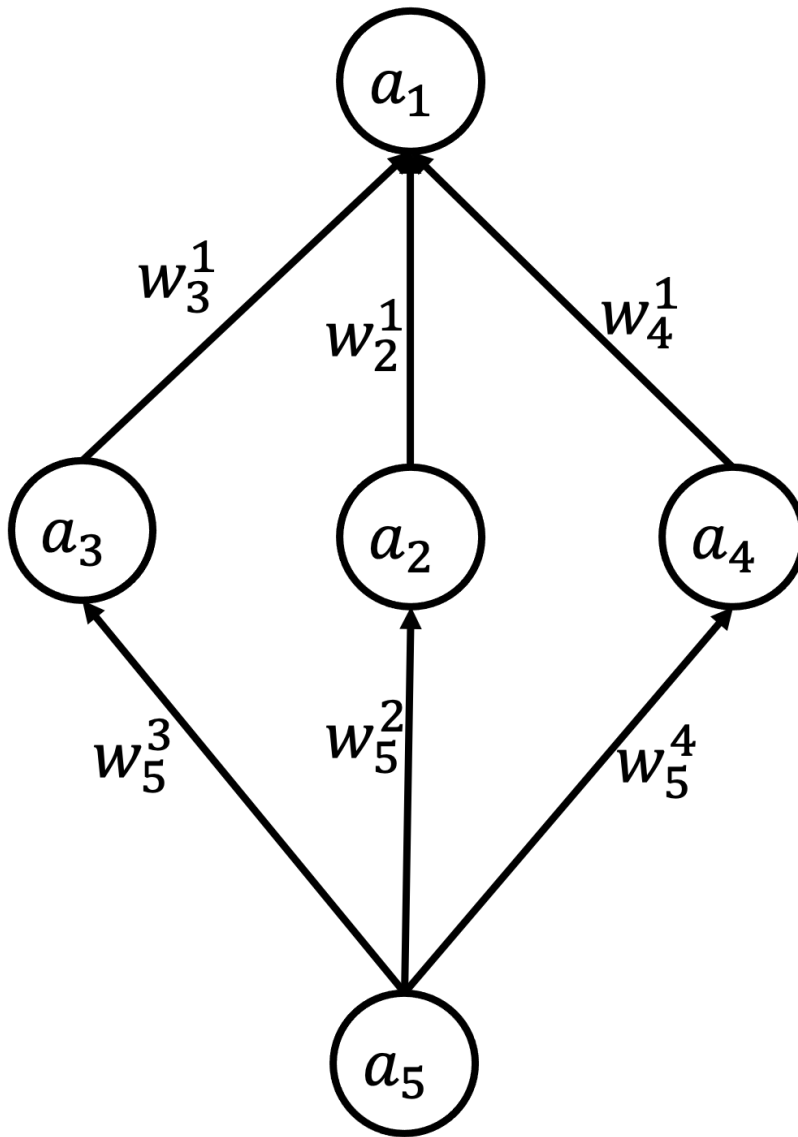
- This is an **individual** assignment. All help from others (from the web, books other than text, or people other than the TA or instructor) must be clearly acknowledged.

Neural Networks

Consider the following neural network:

```
In [2]: from IPython.display import Image  
Image(filename='simple_nn.png', width=400)
```

Out [2]:



where $a_i = \sum_j w_j^i z_j$, $z_i = f_i(a_i)$ for $i = 1, 2, 3, 4$, $z_5 = a_5$ (an input neuron), and $f_1(x) = f_2(x) = f_3(x) = f_4(x) = \text{sigmoid}(x)$.

Part A

(a) Write a function (in math) to simulate the neural network calculating its output.

$$\begin{aligned}
 z_5 &= a_5 \\
 a_2 &= w_5^2 \cdot a_5 \\
 z_2 &= \text{sigmoid}(a_2) \\
 a_3 &= w_5^3 \cdot a_5 \\
 z_3 &= \text{sigmoid}(a_3) \\
 a_4 &= w_5^4 \cdot a_5 \\
 z_4 &= \text{sigmoid}(a_4) \\
 a_1 &= w_2^1 \cdot z_2 + w_3^1 \cdot z_3 + w_4^1 \cdot z_4 \\
 z_1 &= \text{sigmoid}(a_1)
 \end{aligned}$$

(put your math as embedded latex, or otherwise neatly formatted here)

Part b

(b) Deduce the equation to calculate δ_i (the error value per neuron) for all the neurons.

Write a function that given a training sample and the weights of the network calculate δ_i for each neuron.

Hint:

#

SIGMOID

$$f(x) = \frac{1}{1 + e^{-x}}$$

$$df(x)/dx = f(x)(1 - f(x))$$

#

LOSS FUNCTION

$$error = 0.5 * (target - z_1)^2$$

#

Equations for δ_i (error per neuron)

$$\delta_1 = (-target + z_1) \cdot z_1(1 - z_1)$$

$$\delta_i = z_i(1 - z_i) \cdot w_1^i \cdot \delta_1 \quad \text{for } i = 2, 3, 4$$

$$\delta_5 = \delta_2 \cdot w_5^2 + \delta_3 \cdot w_5^3 + \delta_4 \cdot w_5^4$$

Function: compute_deltas((a_5), target, weights)

$$a_2 = w_5^2 \cdot a_5, \quad z_2 = \sigma(a_2)$$

$$a_3 = w_5^3 \cdot a_5, \quad z_3 = \sigma(a_3)$$

$$a_4 = w_5^4 \cdot a_5, \quad z_4 = \sigma(a_4)$$

$$a_1 = w_2^1 \cdot z_2 + w_3^1 \cdot z_3 + w_4^1 \cdot z_4, \quad z_1 = \sigma(a_1)$$

$$\delta_1 = (-target + z_1) \cdot z_1(1 - z_1)$$

$$\delta_2 = z_2(1 - z_2) \cdot w_1^2 \cdot \delta_1$$

$$\delta_3 = z_3(1 - z_3) \cdot w_1^3 \cdot \delta_1$$

$$\delta_4 = z_4(1 - z_4) \cdot w_1^4 \cdot \delta_1$$

$$\delta_5 = \delta_2 \cdot w_5^2 + \delta_3 \cdot w_5^3 + \delta_4 \cdot w_5^4$$

Where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid activation function.

Part c

(c) Assuming that the weight matrix is:

	1	2	3	4
2	3			
3	-4			
4	-1			
5		-3	2	-10

use the functions from items (a) and (b) to calculate the output of each neuron, z_i , and the error, δ_i , for the following training samples:

x	y
0.0	0.5
1.0	0.1

Forward and Backward Pass Results

Outputs z_i and Errors δ_i per Neuron

Input / Target	z_2	z_3	z_4	z_1	δ_2	δ_3	δ_4	δ_1	δ_5
$x = 0.0, y = 0.5$	0.5000	0.5000	0.5000	0.2689	0.0341	0.0454	0.0114	-0.0454	0.2789
$x = 1.0, y = 0.1$	0.0474	0.8808	0.0000	0.0329	0.0003	0.0009	0.0000	-0.0021	0.0033

Parts d and e -- the programming part!

(d) Write a function to train the neural network using the stochastic gradient descent.

(e) Use the function to train the network with the following training samples:

x	y
-3.0	0.7312
-2.0	0.7339
-1.5	0.7438
-1.0	0.7832
-0.5	0.8903
0.0	0.9820
0.5	0.8114
1.0	0.5937
1.5	0.5219
2.0	0.5049
3.0	0.5002

Plot the evolution of the error and the final predictions of the trained network. Write down the weights of the trained network.

```
In [3]: ###Answer:**

import numpy as np

np.random.seed(42)

# defining the backprop for Sigmoid Function
def back_sigmoid(x):
    return x*(1-x)

# defining the Sigmoid Function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def forward_pass(X, weights_input_hidden, weights_hidden_output):
    # Hidden layer linear combination: (N, 1) @ (1, 3) -> (N, 3)
    a_hidden = X @ weights_input_hidden # a_2, a_3, a_4

    # Hidden layer activation (Sigmoid): z_2, z_3, z_4
    hiddenLayer_activations = sigmoid(a_hidden)

    # Output layer linear combination: (N, 3) @ (3, 1) -> (N, 1)
    a_output = hiddenLayer_activations @ weights_hidden_output # a_1

    # Output layer activation (Sigmoid): z_1
    output = sigmoid(a_output) # z_1

    return output, hiddenLayer_activations # z_1, [z_2, z_3, z_4]

def backward_pass(X, y, output, weights_hidden_output, weights_input_hidden, h
    # calculating rate of change of error w.r.t weight between hidden and output

    # Output layer error ( $\delta_1$ )
    delta_1 = (output - y) * back_sigmoid(output) # shape: (N, 1)

    # Gradient for output layer's weights
    # hiddenLayer_activation.T: (3, N), delta_1: (N, 1) -> error_wrt_weights_hidden_output
    error_wrt_weights_hidden_output = hiddenLayer_activations.T @ delta_1 #  $\delta_1$ 

    # Hidden layer errors ( $\delta_2, \delta_3, \delta_4$ )
    # delta_1 @ weights_hidden_output.T: (N, 1) @ (1, 3) -> (N, 3)
    delta_hidden = back_sigmoid(hiddenLayer_activations) * (delta_1 @ weights_hidden_output.T)

    # Gradient for hidden layer's weights
    # X.T: (1, N), delta_hidden: (N, 3) -> error_wrt_weights_input_hidden: (1, N)
    error_wrt_weights_input_hidden = X.T @ delta_hidden

    return error_wrt_weights_hidden_output, error_wrt_weights_input_hidden

def train(X_train, y_train):

    # defining the model architecture
    inputLayer_neurons = 1 # number of neurons at input
```

```

hiddenLayer_neurons = 3 # number of hidden layers neurons
outputLayer_neurons = 1 # number of neurons at output layer

# initializing weight
weights_input_hidden = np.random.uniform(size=(inputLayer_neurons, hiddenLayer_neurons))
weights_hidden_output = np.random.uniform(
    size=(hiddenLayer_neurons, outputLayer_neurons)
)
# defining the parameters
lr = 0.1
epochs = 1000

losses = []

for ep in range(epochs):
    epoch_losses = []

    for i in range(X_train.shape[0]):
        x = X_train[i].reshape(1, 1)
        y = y_train[i].reshape(1, 1)

        output_, hiddenLayer_activations = forward_pass(x, weights_input_hidden, weights_hidden_output)

        loss = np.square(y - output_) / 2
        epoch_losses.append(loss.item())

    ## Backward Propagation
    error_wrt_weights_hidden_output, error_wrt_weights_input_hidden = backward_pass(output_, hiddenLayer_activations, y, weights_hidden_output, weights_input_hidden)

    # updating the weights
    weights_hidden_output -= lr * error_wrt_weights_hidden_output
    weights_input_hidden -= lr * error_wrt_weights_input_hidden

    avg_loss = np.mean(epoch_losses)
    losses.append(avg_loss)
    if ep % 100 == 0:
        print(f"Epoch {ep}: Loss = {avg_loss:.5f}")

import matplotlib.pyplot as plt
plt.plot(losses)
plt.show()

final_pred, _ = forward_pass(X_train, weights_input_hidden, weights_hidden_output)

plt.plot(final_pred, c='r')
plt.plot(y_train, c='b')
plt.show()

print("Weights of the network are: ")
print("w13, w12, w14", weights_hidden_output.T)
print("w35, w25, w45", weights_input_hidden)

# defining training data
X_train = np.zeros((11, 1)).astype(np.float32)
y_train = np.zeros((11, 1)).astype(np.float32)

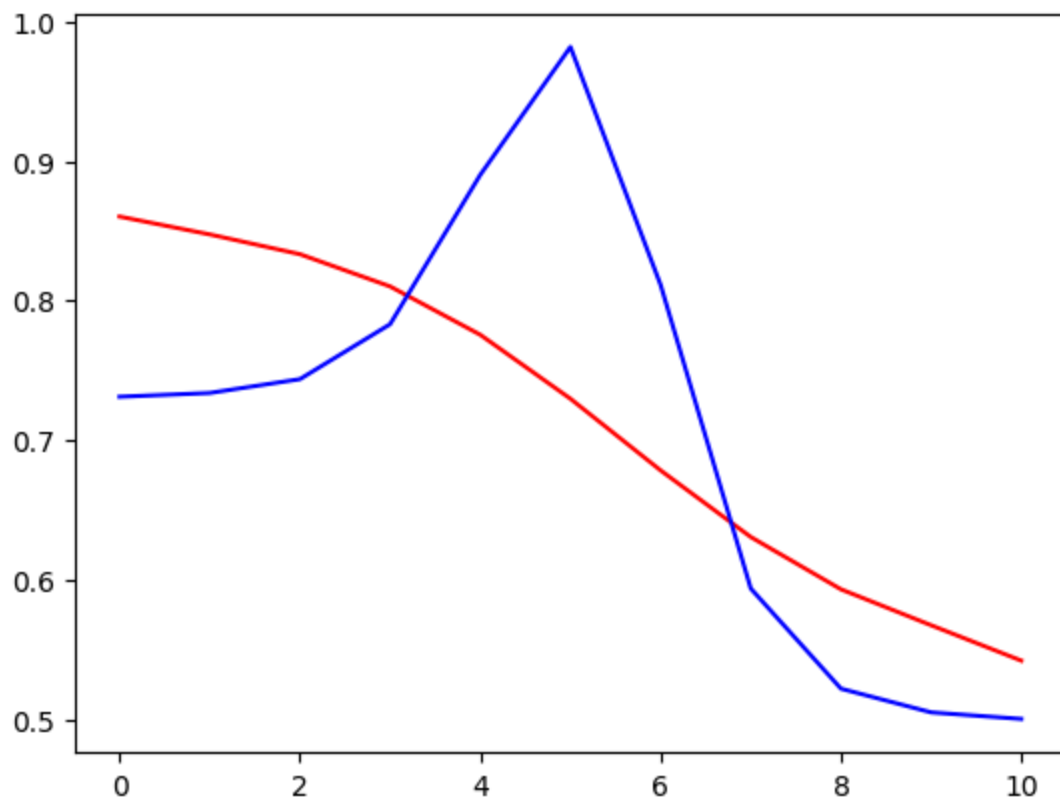
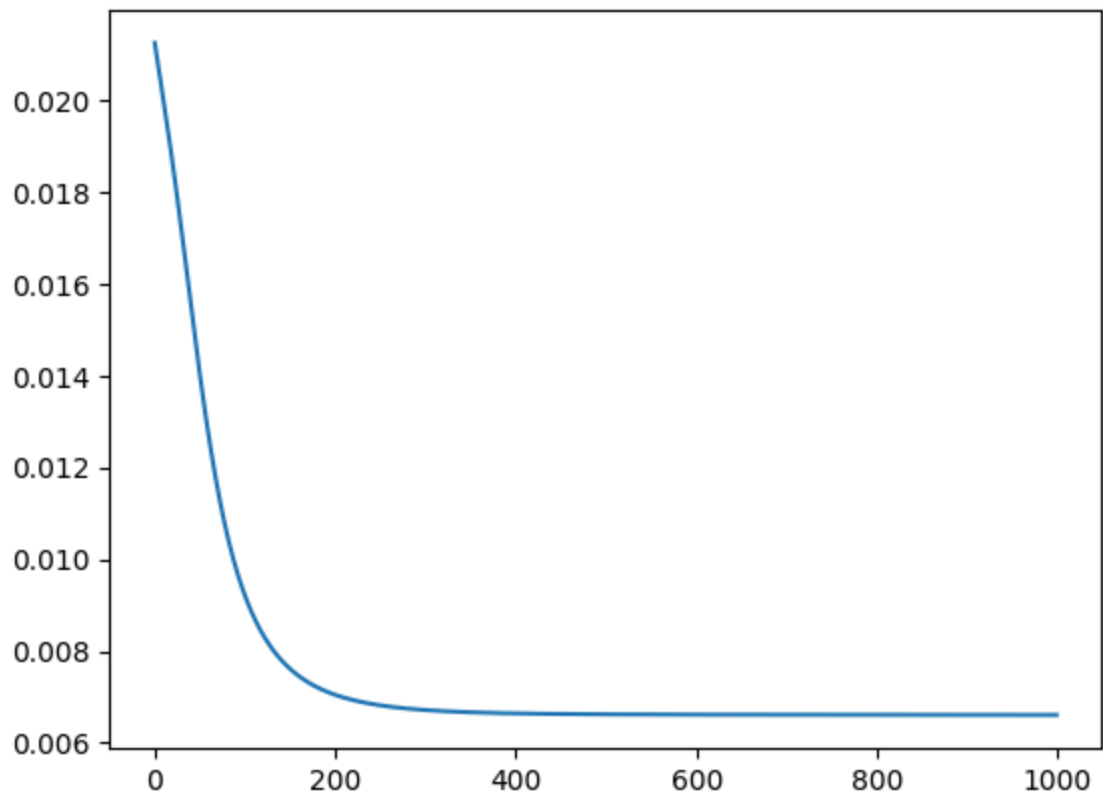
```

```
X_train[0] = -3
X_train[1] = -2
X_train[2] = -1.5
X_train[3] = -1.0
X_train[4] = -0.5
X_train[5] = 0.0
X_train[6] = 0.5
X_train[7] = 1.0
X_train[8] = 1.5
X_train[9] = 2.0
X_train[10] = 3.0
```

```
y_train[0] = 0.7312
y_train[1] = 0.7339
y_train[2] = 0.7438
y_train[3] = 0.7832
y_train[4] = 0.8903
y_train[5] = 0.9820
y_train[6] = 0.8114
y_train[7] = 0.5937
y_train[8] = 0.5219
y_train[9] = 0.5049
y_train[10] = 0.5002
```

```
train(X_train, y_train)
```

```
Epoch 0: Loss = 0.02124
Epoch 100: Loss = 0.00920
Epoch 200: Loss = 0.00704
Epoch 300: Loss = 0.00671
Epoch 400: Loss = 0.00664
Epoch 500: Loss = 0.00662
Epoch 600: Loss = 0.00661
Epoch 700: Loss = 0.00661
Epoch 800: Loss = 0.00661
Epoch 900: Loss = 0.00661
```



Weights of the network are:

w13, w12, w14 $\begin{bmatrix} 1.847739 & -0.0644739 & 0.20371785 \end{bmatrix}$

w35, w25, w45 $\begin{bmatrix} -1.12331086 & 0.87973316 & 0.50597873 \end{bmatrix}$

In []: